

ÉCOLE DES SCIENCES DE L'INFORMATION

Programmation Avancée en C

LIVRABLE 2

Dossier de Conception Technique

Système de Détection de Collision pour Essaim Autonome (UAV)

Projet Industriel — 2025/2026

1. Présentation Générale du Système

Le système développé répond à une problématique industrielle critique : la détection en temps réel de la paire de drones la plus proche dans un essaim de 10 000 micro-drones autonomes. Chaque milliseconde, les capteurs radars génèrent un flux de coordonnées spatiales 3D (x, y, z) qu'il faut analyser instantanément pour prévenir tout crash en chaîne.

1.1 Objectifs du Projet

- Identifier la paire de drones en collision imminente en temps réel
- Respecter les contraintes mémoire d'un processeur embarqué
- Interdire toute indexation par crochets — arithmétique de pointeurs exclusive
- Atteindre une complexité $O(n \log n)$ — contre $O(n^2)$ pour l'approche naïve

1.2 Architecture Générale

Le système est structuré en quatre sections fonctionnelles au sein d'un fichier source unique :

Section	Responsabilité	Complexité
Structures de données	Topologie Drone + PaireCritique	$O(1)$
Module de tri spatial	QuickSort par axe X sur pointeurs	$O(n \log n)$
Détection de collision	Fenêtre glissante avec pruning	$O(n)$ moy.
Point d'entrée	Init, mesure, rapport d'alerte	$O(n)$

2. Topologie des Données en Mémoire

2.1 Structure Hétérogène Drone

Conformément au cahier des charges (Contrainte §1), la structure hétérogène est définie ainsi :

```
typedef struct {  
    int    id;        /* Identifiant unique  - 4 octets (int 32-bit)  */  
    float x;          /* Coordonnée axe X    - 4 octets (float 32-bit) */  
    float y;          /* Coordonnée axe Y    - 4 octets (float 32-bit) */  
    float z;          /* Coordonnée axe Z    - 4 octets (float 32-bit) */  
} Drone;             /* Taille totale : 16 octets (sans padding ici) */
```

Le bilan mémoire de la flotte complète est le suivant :

Composant	Calcul	Taille
Un drone (struct)	4 + 4 + 4 + 4 octets	16 octets
Bloc essaim (heap)	16 × 10 000	160 000 octets ≈ 156 KB
Tableau d'indices (tri)	8 × 10 000 (pointeurs 64-bit)	80 000 octets ≈ 78 KB
Total RAM utilisée	—	≈ 234 KB

2.2 Allocation Dynamique — Bloc Continu

Conformément à la Contrainte §2, tous les drones sont alloués en un seul bloc mémoire contigu sur le tas (heap) :

```
Drone *essaim = (Drone *)malloc(N_DRONES * sizeof(Drone));
```

Ce choix architectural est fondamental. La contiguïté mémoire garantit la localité de cache (cache locality), réduisant significativement les cache-miss lors du parcours séquentiel. Les adresses des drones sont prévisibles :

```
Adresse du drone i = base + i × sizeof(Drone)  
                  = base + i × 16  (octets)
```

3. Arithmétique Pure de Pointeurs

3.1 Principe Fondamental

La contrainte la plus stricte du cahier des charges interdit formellement l'opérateur d'indexation `[]` dans tout le code. Toute navigation dans le tableau doit s'effectuer via l'arithmétique des pointeurs. La règle de base en C est :

```
(Type *)ptr + k  ≡  adresse(ptr) + k × sizeof(Type)
```

Cette équivalence est garantie par la norme ISO C11. Les deux notations `ptr[k]` et `*(ptr + k)` sont sémantiquement identiques, mais la seconde est imposée ici comme discipline d'ingénierie.

3.2 Table de Correspondance — Interdit vs Autorisé

Opération	INTERDIT (indexation)	AUTORISÉ (arithmétique)
Lire le i-ième drone	essaim[i]	*(essaim + i)
Obtenir un pointeur vers i	&essaim[i]	essaim + i
Lire le champ x du drone i	essaim[i].x	(essaim + i)->x
Lire le j-ième pointeur	index[j]	*(index + j)
Écrire dans index[j]	index[j] = p	*(index + j) = p

3.3 Exemples dans le Code

Initialisation de la flotte

```
for (int i = 0; i < n; i++) {
    Drone *p = essaim + i;    /* Pointeur vers le i-ième drone */
    p->id     = i + 1;
    p->x      = rand_float(-5000.0f, 5000.0f);
}
```

Construction du tableau d'indices

```
for (int i = 0; i < n; i++) {
    *(index + i) = essaim + i; /* Écriture par déréférencement */
}
```

Échange de pointeurs dans QuickSort

```
static void echanger_pointeurs(Drone **a, Drone **b) {
    Drone *tmp = *a;
    *a = *b;
    *b = tmp;
}

/* Appel : echanger_pointeurs(pp + i, pp + j); */
```

4. Module de Tri Spatial — QuickSort par Axe X

4.1 Pourquoi Trier par Axe X ?

L'idée centrale de l'optimisation est géométrique : si deux drones sont très proches en distance 3D, ils doivent nécessairement être proches selon chacun des trois axes. En triant par l'axe X, on peut exploiter ce fait pour éliminer rapidement les paires impossibles sans calculer leur distance complète.

4.2 Tableau d'Indirection

Plutôt que de déplacer les structures Drone de 16 octets lors des échanges du tri, on trie un tableau auxiliaire de pointeurs (8 octets chacun). Avantage : le coût de chaque échange est réduit de moitié, et le bloc mémoire principal reste intact.

```
Drone **index = (Drone **)malloc(n * sizeof(Drone *));
/* index[0..n-1] = tableau de pointeurs triés par ->x */
/* Les drones eux-mêmes ne bougent pas en mémoire */
```

4.3 Partition de Lomuto

La partition de Lomuto sélectionne le dernier élément comme pivot. Elle réorganise le sous-tableau pour que tous les éléments dont $x \leq \text{pivot}_x$ soient à gauche du pivot, et les autres à droite :

```
static int partitionner(Drone **pp, int taille) {
    float pivot_x = (*(pp + taille - 1))->x;
    int i = -1;
    for (int j = 0; j < taille - 1; j++) {
        if ((*(pp + j))->x <= pivot_x) {
            i++;
            echanger_pointeurs(pp + i, pp + j);
        }
    }
    echanger_pointeurs(pp + i + 1, pp + taille - 1);
    return i + 1;
}
```

5. Algorithme de Détection — Fenêtre Glissante

5.1 Principe du Sweep Line

Après tri par X, les drones sont ordonnés selon leur coordonnée horizontale. On parcourt ce tableau trié avec deux indices i et j . Pour chaque drone i , on compare avec $j > i$ en avançant vers la droite jusqu'à ce que la condition de pruning soit vérifiée.

5.2 Pruning Géométrique — Clé de l'Optimisation

La sortie anticipée de la boucle interne est la clé algorithmique du système. La condition est :

```
float delta_x = dj->x - di->x;
if (delta_x * delta_x >= resultat.dist_sq) break;
```

Justification : Après tri, $dj \rightarrow x \geq di \rightarrow x$. Si la distance sur le seul axe X dépasse déjà la meilleure distance 3D connue, alors la distance 3D complète sera nécessairement supérieure aussi — quelle que soient les coordonnées Y et Z. On arrête immédiatement sans aucun calcul inutile.

5.3 Évitement du sqrt() Coûteux

La comparaison de distances conserve son ordre sans extraction de racine carrée :

$$d1 < d2 \iff d1^2 < d2^2 \quad (\text{pour } d1, d2 \geq 0)$$

On stocke donc `dist_sq` (distance au carré) tout au long de l'algorithme, et on n'appelle `sqrt()` qu'une seule fois à la toute fin, pour l'affichage du résultat. Un appel `sqrt()` coûte environ 20 cycles CPU contre 3 cycles pour une multiplication flottante.

5.4 Structure PaireCritique

```
typedef struct {
    Drone *drone_a; /* Pointeur vers le 1er drone critique */
    Drone *drone_b; /* Pointeur vers le 2e drone critique */
    float dist_sq; /* Distance² - sqrt uniquement en fin */
} PaireCritique;
```

6. Sécurité Mémoire et Robustesse

6.1 Vérification des Allocations

Chaque appel malloc est suivi d'une vérification explicite. En environnement embarqué, une allocation échouée doit être gérée immédiatement :

```
Drone *essaim = (Drone *)malloc(N * sizeof(Drone));
if (!essaim) {
    fprintf(stderr, "[ERREUR CRITIQUE] Allocation échouée.\n");
    return EXIT_FAILURE;
}
```

6.2 Politique de Libération Mémoire

Pointeur	Alloué dans	Libéré dans	Garantie
essaim	main()	main() avant return	Libération unique
index	detecter_paire_critique()	avant return de la fonction	Libération unique

6.3 Absence de VLA

Aucun tableau à longueur variable (Variable Length Array — VLA) n'est utilisé. Cette conformité est requise pour les compilateurs embarqués qui désactivent souvent cette extension optionnelle de C11.

7. Compilation et Exécution

7.1 Commande de Compilation

```
gcc -Wall -Wextra -O2 -std=c11 uav_collision.c -lm -o uav
```

Option	Signification
-Wall -Wextra	Activer tous les avertissements du compilateur
-O2	Optimisation niveau 2 — équilibre vitesse / taille
-std=c11	Conformité au standard ISO C11
-lm	Liaison avec la bibliothèque mathématique (sqrtf)

7.2 Exemple de Sortie

```
[ MEM ] Bloc alloué : 10000 drones x 16 octets = 156 KB
[ INIT ] Chargement de 10000 drones en mémoire...
[ SCAN ] Analyse de collision en cours (N=10000)...
```

⚠ ALERTE COLLISION IMMINENTE ⚠

Drone A : ID= 5508 pos=(-4420.38, 780.87, 3109.35)

Drone B : ID= 9603 pos=(-4414.61, 776.44, 3109.80)

Distance minimale : 7.286341 mètres

Temps d'exécution : 1.08 ms

Complexité : $O(n \log n)$